

简书

首页

下载APP

会员

IT技术

搜索

Q

Aa

beta

登录

注册



浮华_du

关注

总资产16

flutter-isolate详解



浮华_du

关注

IP属地: 北京

0.939 2021.06.25 14:08:11 字数 1,124 阅读 5,173



一. isolate简介

赏

是单线程，Dart 为我们提供了 isolate，isolate 跟线程差不多，它可以理解为 Dart 中的线程。isolate 与线程的区别就是线程与线程之间是共享内存的，而 isolate 和 isolate 之间是内存不共享的，所以叫 isolate (隔离)。因此也不存在锁竞争问题，两个 Isolate 完全是两条独立的执行线，且每个 Isolate 都有自己的事件循环，它们之间只能通过发送消息通信，所以它的资源开销低于线程。

原文链接：<https://blog.csdn.net/u011578734/article/details/108853613>

大多数计算机中，甚至在移动平台上，都在使用多核 CPU。为了有效利用多核性能，开发者一般使用共享内存的方式让线程并发地运行。然而，多线程共享数据通常会导致很多潜在的问题，并导致代码运行出错。

为了解决多线程带来的并发问题，Dart 使用 isolates 替代线程，所有的 Dart 代码均运行在一个 isolates 中。每一个 isolates 有它自己的堆内存以确保其状态不被其它 isolates 访问。

每个 isolate 都拥有自己的事件循环及队列（MicroTask 和 Event）。这意味着在一个 isolate 中运行的代码与另外一个 isolate 不存在任何关联。

isolate 是 Dart 对 actor 并发模式的实现。运行中的 Dart 程序由一个或多个 actor 组成，这些 actor 也就是 Dart 概念里面的 isolate。isolate 是有自己的内存和单线程控制的运行实体。isolate 本身的意思是“隔离”，因为 isolate 之间的内存存在逻辑上是隔离的。isolate 中的代码是按顺序执行的，任何 Dart 程序的并发都是运行多个 isolate 的结果。因为 Dart 没有共享内存的并发，没有竞争的可能性所以不需要锁，也就不担心死锁的问题。

二. isolate 与 async 关系:

之前介绍过 async/await Future 的原理, async 关键字实现了异步操作, 而所谓的异步其实也是运行在同一线程中并没有开启新的线程, 只是通过单线程的任务调度实现一个先执行其他的代码片段, 等这边有结果后再返回的异步效果.

- Isolate 可以实现异步并行多个任务
- Future 实现异步串行多个任务

举个栗子:

假设有一个任务, 需要计算 1+2+...100 的和
我们通常会这么写:

写下你的评论...

评论 2

赞 12

...

Banner/flutter_swiper 重叠

阅读 230

iOS-嵌套滚动

阅读 914

热门故事

前世渣男把我迷晕后送到新科状元床上!

你看过最恐怖的悬疑小说是哪部?

妖艳女主播专挑凶宅直播跳舞结果.....

我捡回来的奶狗弟弟变身狼狗! 遭不住

穿成王宝钏: 做大唐第一渣女

推荐阅读

Java 实现异步编程的 8 种方式

阅读 359

如何让三个线程按顺序执行?

阅读 860

JAVA 集合面试题

阅读 276

Java 线程<第四篇>: Hook 线程以及捕获线程执行异常

阅读 309

[iOS] 循环网络请求顺序执行 (信号量)

阅读 400

```

1
2 class IsolateTestPage extends StatefulWidget {
3   @override
4   State<StatefulWidget> createState() {
5     return IsolateTestPageState();
6   }
7 }
8
9 class IsolateTestPageState extends State<IsolateTestPage> {
10   var content = "点击计算按钮,开始计算";
11
12   @override
13   Widget build(BuildContext context) {
14     return Scaffold(
15       appBar: AppBar(
16         title: Text("Isolate"),
17       ),
18       body: SafeArea(
19         child: Center(
20           child: Column(
21             children: <Widget>[
22               Container(
23                 width: double.infinity, height: 500, child: Text(content)),
24               TextButton(
25                 child: Text('计算'),
26                 onPressed: () {
27                   int result = sum(100);
28                   content = "总和$result";
29                   setState(() {});
30                 },
31             ),
32           ],
33         ),
34       ),
35     );
36   }
37
38   //计算0到 num 数值的总和
39   int sum(int num) {
40     int count = 0;
41     while (num > 0) {
42       count = count + num;
43       num--;
44     }
45     return count;
46   }
47 }

```

试一下,结果是可以的;但是如果我们改为计算1+2+...100000000000的和,运行代码会发现,直接卡死了

这时候就需要创建一个isolate来执行这个耗时的任务,而不影响其他任务的执行. 那么首先,如何创建一个isolate?

三.创建isolate 及 isolate通信

- (1).获取当前main isolate的ReceivePort及SendPort
- (2).使用Isolate.spawn创建新的isolate,需要传入新的isolate需要完成的任务名称及创建者(main isolate)的sendPort. (用于将新的isolate的sendPort传递给创建者)
- (3).在任务方法中,获取新的isolate的ReceivePort及SendPort
- (4).将新的isolate的SendPort,通过main isolate的sendPort,发送给main isolate,使新的isolate的SendPort能在main isolate中发送消息
- (5). SendPort发送消息, ReceivePort接收消息,互相通信

1. receivePort.listen((message) {}) 能收到其对应的sendPort发送的

消息

```

1  _testIsolate() async {
2      ReceivePort rp1 = new ReceivePort();
3      SendPort port1 = rp1.sendPort;
4      // 通过spawn新建一个isolate, 并绑定静态方法
5      Isolate newIsolate = await Isolate.spawn(doWork, port1);
6
7      SendPort port2;
8      rp1.listen((message) {
9          print("rp1 收到消息: $message"); //2. 4. 7.rp1收到消息
10         if (message[0] == 0) {
11             port2 = message[1]; //得到rp2的发送器port2
12         } else {
13             if (port2 != null) {
14                 print("port2 发送消息");
15                 port2?.send([1, "这条信息是 port2 在main isolate中 发送的"]); // 8.port2发送消息
16             }
17         }
18     });
19
20     print("port1--main isolate发送消息");
21     port1.send([1, "这条信息是 port1 在main isolate中 发送的"]); //1.port1发送消息
22
23     // newIsolate.kill();
24 }
25
26 // 新的isolate中可以处理耗时任务
27 static void doWork(SendPort port1) {
28     ReceivePort rp2 = new ReceivePort();
29     SendPort port2 = rp2.sendPort;
30     rp2.listen((message) {
31         //9.10 rp2收到消息
32         print("rp2 收到消息: $message");
33     });
34     // 将新isolate中创建的SendPort发送到main isolate中用于通信
35     print("port1--new isolate发送消息");
36     port1.send([0, port2]); //3.port1发送消息,传递[0,rp2的发送器]
37     // 模拟耗时5秒
38     sleep(Duration(seconds: 5));
39     print("port1--new isolate发送消息");
40     port1.send([1, "这条信息是 port1 在new isolate中 发送的"]); //5.port1发送消息
41     print("port2--new isolate发送消息");
42     port2.send([1, "这条信息是 port2 在new isolate中 发送的"]); //6.port2发送消息
43 }
44
45 //I/flutter (14639): port1--main isolate发送消息
46 //I/flutter (14639): rp1 收到消息: [1, 这条信息是 port1 在main isolate中 发送的]
47 //I/flutter (14639): port1--new isolate发送消息
48 //I/flutter (14639): rp1 收到消息: [0, SendPort]
49 //I/flutter (14639): port1--new isolate发送消息
50 //I/flutter (14639): port2--new isolate发送消息
51 //I/flutter (14639): rp1 收到消息: [1, 这条信息是 port1 在new isolate中 发送的]
52 //I/flutter (14639): port2 发送消息
53 //I/flutter (14639): rp2 收到消息: [1, 这条信息是 port2 在new isolate中 发送的]
54 //I/flutter (14639): rp2 收到消息: [1, 这条信息是 port2 在main isolate中 发送的]
55

```

2. dynamic result = await receivePort.first; 只能收到第一条消息

```

1  _testIsolate() async {
2      ReceivePort rp1 = new ReceivePort();
3      SendPort port1 = rp1.sendPort;
4      // 通过spawn新建一个isolate, 并绑定静态方法
5      Isolate newIsolate = await Isolate.spawn(doWork, port1);
6
7      SendPort port2;
8      dynamic receiveMsg = await rp1.first; //只拿到第一条收到结果
9      print('rp1 收到消息--$receiveMsg');
10     if (receiveMsg is SendPort) {
11         SendPort port2 = receiveMsg;

```

```

12 // print('rp1 收到消息--port2');
13 port2.send([1, "这条信息是 port2 在main isolate中 发送的"]);
14 }
15
16 // newIsolate.kill();
17 }
18
19 // 新的isolate中可以处理耗时任务
20 static void doWork(SendPort port1) {
21   ReceivePort rp2 = new ReceivePort();
22   SendPort port2 = rp2.sendPort;
23   rp2.listen((message) {
24     print("rp2 收到消息-- $message");
25   });
26   // 将新isolate中创建的SendPort发送到main isolate中用于通信
27   print("port1--new isolate发送消息--port2");
28   port1.send(port2);
29   // 模拟耗时5秒
30   sleep(Duration(seconds: 5));
31   print("port1--new isolate发送消息--啊哈哈");
32   port1.send("啊哈哈");
33 }

```

3.解决示例问题

知道怎么创建isolate了,我们再看怎么来计算1+2+...100000000000的和

(1)使用listen监听,结果回调的形式

1.创建isolate

2.打通两个isolate的通道(能互相发送消息)

3.main isolate将要计算的最大数传递给new isolate; newisolate计算,计算完成后,将结果发送回 main isolate

```

1 calculation(int n, Function(int result) success) async {
2   //创建一个ReceivePort
3   final receivePort1 = new ReceivePort();
4   //创建isolate
5   Isolate isolate = await Isolate.spawn(createIsolate, receivePort1.sendPort);
6   receivePort1.listen((message) {
7     if (message is SendPort) {
8       SendPort sendPort2 = message;
9       sendPort2.send(n);
10    } else {
11      print(message);
12      success(message);
13    }
14  });
15 }
16
17 //创建isolate必须需要的参数
18 static void createIsolate(SendPort sendPort1) {
19   final receivePort2 = new ReceivePort();
20   //绑定
21   print("sendPort1发送消息--sendPort2");
22   sendPort1.send(receivePort2.sendPort);
23   //监听
24   receivePort2.listen((message) {
25     //获取数据并解析
26     print("receivePort2接收到消息--$message");
27     if (message is int) {
28       num result = summ(message);
29       sendPort1.send(result);
30     }
31   });
32 }
33
34 //计算0到 num 数值的总和

```

```

35 | static num summ(int num) {
36 |     int count = 0;
37 |     while (num > 0) {
38 |         count = count + num;
39 |         num--;
40 |     }
41 |     return count;
42 | }

```

(2)receivePort.first只能收到第一条消息; 我们可以再创建一个ReceivePort 用来传递消息,如下:

```

1 | static Future<dynamic> calculation(int n) async {
2 |     //创建一个ReceivePort
3 |     final receivePort1 = new ReceivePort();
4 |     //创建isolate
5 |     Isolate isolate = await Isolate.spawn(createIsolate, receivePort1.sendPort);
6 |
7 |     //使用 receivePort1.first 获取sendPort1发送来的数据
8 |     final sendPort2 = await receivePort1.first as SendPort;
9 |     print("receivePort1接收到消息--sendPort2");
10 |    //接收消息的ReceivePort
11 |    final answerReceivePort = new ReceivePort();
12 |    print("sendPort2发送消息--[$n,answerSendPort]");
13 |    sendPort2.send([n, answerReceivePort.sendPort]);
14 |    //获得数据并返回
15 |    num result = await answerReceivePort.first;
16 |    print("answerReceivePort接收到消息--计算结果$result");
17 |    return result;
18 | }
19 |
20 | //创建isolate必须需要的参数
21 | static void createIsolate(SendPort sendPort1) {
22 |     final receivePort2 = new ReceivePort();
23 |     //绑定
24 |     print("sendPort1发送消息--sendPort2");
25 |     sendPort1.send(receivePort2.sendPort);
26 |     //监听
27 |     receivePort2.listen((message) {
28 |         //获取数据并解析
29 |         print("receivePort2接收到消息--$message");
30 |         final n = message[0] as num;
31 |         final send = message[1] as SendPort;
32 |         //返回结果
33 |         num result = summ(n);
34 |         print("answerSendPort发送消息--计算结果$result");
35 |         send.send(result);
36 |     });
37 | }
38 |
39 | //计算0到 num 数值的总和
40 | static num summ(int num) {
41 |     int count = 0;
42 |     while (num > 0) {
43 |         count = count + num;
44 |         num--;
45 |     }
46 |     return count;
47 | }

```

4. isolate的暂停 恢复 结束

```

1 | //恢复 isolate 的使用
2 | isolate.resume(isolate.pauseCapability);
3 |
4 | //暂停 isolate 的使用
5 | isolate.pause(isolate.pauseCapability);
6 |
7 | //结束 isolate 的使用

```

```
8 | isolate.kill(priority: Isolate.immediate);
9 |
10 | //赋值为空 便于内存及时回收
11 | isolate = null;
```

四.flutter中创建isolate---compute()方法

```
1 | TextButton(
2 |   child: Text('flutter创建isolate'),
3 |   onPressed: () async {
4 |     num result = await compute(summ, 10000000000);
5 |     content = "计算结果$result";
6 |     setState(() {});
7 |   },
8 | ),
```

到这里,希望大家对isolate有更深一步的了解.欢迎点点小心心哦~

<https://cloud.tencent.com/developer/article/1676442>



更多精彩内容，就在简书APP



"小礼物走一走，来简书关注我"

赞赏支持

还没有人赞赏，支持一下



浮华_du 仅用于个人学习记录~

总资产16 共写了3.4W字 获得147个赞 共27个粉丝

关注

写下你的评论...

全部评论 2 只看作者

按时间倒序 按时间正序



零陵上将刑道荣大将军 IP属地: 重庆

3楼 11.26 17:02

第一次接触isolate，感觉写的很好。



NetWork小贱 IP属地: 北京

2楼 10.14 20:29

写的太绕了，什么新的isolate 啥的



被以下专题收入，发现更多相似内容



flutter

推荐阅读

更多精彩内容 >

Dart的语法详解系列篇（四）-- 泛型、异步、库等有关详解

版权声明：本文为博主原创文章，未经博主允许不得转载。

<https://www.jianshu.com/p/a4a9c...>



AWeiLoveAndroid 阅读 9,370 评论 0 赞 35



泛型、异步、库
等有关详解

Flutter异步编程详解

不知道大家有没有一个疑问：Dart是单线程执行，那它是如何实现异步操作的呢？本文将提供对Dart/Flutter提供...



chonglingliu 阅读 987 评论 0 赞 2

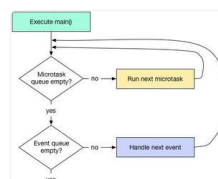


深入理解Flutter多线程

Flutter默认是单线程任务处理的，如果不开启新的线程，任务默认在主线程中处理。事件队列 和iOS应用很像，在...



霸拥_f357 阅读 253 评论 0 赞 1



Flutter 异步编程：Future、Isolate 和事件循环

原文地址：Futures - Isolates - Event Loop原作者：www.didierboelen...



绿豆粥与茶叶蛋 阅读 6,877 评论 1 赞 15

表情管理

表情是什么，我认为表情就是表现出来的情绪。表情可以传达很多信息。高兴了当然就笑了，难过就哭了。两者是相互影响密不可...



Persistenc_6aea 阅读 111,805 评论 2 赞 7

Substrate的transaction-payment模块分析

Substrate的transaction-payment模块分析 transaction-payment模块提供...



建怀 阅读 7,482 评论 0 赞 4

2019-11-28 173

16宿命：用概率思维提高你的胜算 以前的我是风险厌恶者，不喜欢去冒险，但是人生放弃了冒险，也就放弃了无数的可能。 ...



yichen大刀 阅读 5,345 评论 0 赞 4